NETWORK INTRUSION ANALYSIS

PROJECT DOCUMENTATION

Table Of Contents

Table Of Contents 2

1.0 Introduction 4

1.1 Data Description 4

1.2 Assumptions 5

1.3 Hypothesis 6

1.4 Objective 6

2.0 Data Preparation 7

2.1 Data import 7

2.2 Cleaning / Pre-processing 8

2.3 Data Validation 11

3.0 Data Analysis 13

3.1 ANALYSIS 3-1: What is the distribution of connection duration and source bytes across normal and attack traffic? 13

3.2 ANALYSIS 3-2: What is the relationship between connection duration and source bytes? 18

3.3 ANALYSIS 3-3: Is there a significant difference in connection duration and source bytes between attack and normal traffic? 21

3.4 ANALYSIS 3-4: Can connection duration, byte-flow characteristics, TTL behaviour, protocol type, and connection state collectively predict whether a network flow is an attack or normal traffic? 23

4.0 Conclusion 32

4.1 Overall discussion on the findings 32

4.2 Recommendation 32

4.3 Limitation and future direction 32

5.0 References 33

Figure Table

Figure 2.1.1: Importing dataset csv file 7
Figure 2.1.2: Viewing the imported dataset 7
Figure 2.2.1: Removing unwanted characters and converting data types 8
Figure 2.2.2: Inspecting the summary of selected variables 8
Figure 2.2.3: Handling NA values, and using Median and Mode imputation 9
Figure 2.2.4: Script for displaying categories, importing library, and collapsing infrequent categories 9
Figure 2.2.5: Showcase of the proto & state categories before collapsing 10
Figure 2.2.6: Showcase of the proto & state categories after collapsing 10
Figure 2.3.1: Script for data validation 11
Figure 2.3.2: Checking for negative values 11
Figure 2.3.3: Verifying label categories 12
Figure 2.3.4: Final verification of selected variables 12
Figure 3.1: Importing libraries required for data analysis 13
Figure 3.1.1: Script for summary statistics for dur & sbytes. 14
Figure 3.1.2: Output of summary statistics for sbytes & dur. 14
Figure 3.1.3: Script to show histogram for dur. 15
Figure 3.1.4: Histogram for Connection Duration. 15
Figure 3.1.5: Script to show boxplot for dur. 16
Figure 3.1.6: Boxplot for Connection Duration. 16
Figure 3.1.7: Script to show boxplot for sbytes. 16
Figure 3.1.8: Boxplot for Source Bytes. 17
Figure 3.1.9: Script to show density plot for sbytes. 17
Figure 3.1.10: Density plot for Source Bytes. 18
Figure 3.2.1: Script to show scatter plot 19
Figure 3.2.2: Scatter Plot for Duration and Source Bytes 19
Figure 3.2.3: Script to show Spearman's Correlation Coefficient 19
Figure 3.2.4: Output for Spearman's Correlation Coefficient 20
Figure 3.2.5: Script to show Correlation Heatmap 20
Figure 3.2.6: Correlation Heatmap for Duration and Source Bytes 20
Figure 3.3.1: Script to show T-Tests 21
Figure 3.3.2: Output of T-Tests 22
Figure 3.3.3: Script to show Wilcoxon Rank-Sum Tests 22
Figure 3.3.4: Output for Wilcoxon Rank-Sum Tests 22
Figure 3.4.1: Script for Logistic Regression 24
Figure 3.4.2: Initial Logistic Regression Model 24
Figure 3.4.3: Coefficients for Initial Logistic Regression Model 25
Figure 3.4.4: Confusion Matrix and Statistics for Initial Logistic Regression Model 25
Figure 3.4.5: Final Logistic Regression Model 26
Figure 3.4.6: Coefficients for Final Logistic Regression Model 26
Figure 3.4.7: Confusion Matrix and Statistics for Final Logistic Regression Model 27
Figure 3.4.8: Script for Random Forest 28
Figure 3.4.9: Random Forest Model 29
Figure 3.4.10: Importance of the selected variables 29
Figure 3.4.11: Variable Important Plot of selected variables 29
Figure 3.4.12: Confusion Matrix and Statistics for Random Forest Model 30

1.0 Introduction

1.1 Data Description

The dataset used in this project is a subset of the UNSW-NB15 network intrusion detection dataset, containing 175,342 network flow records. Every record reflects a single bi-directional network connection created under scenarios of simulated attack and normal traffic. The dataset has approximately 49 features of continuous and categorical variables that are frequently used in intrusion detection and network traffic analytics.

Below are the descriptions of the variables used for this study:

Target Variable

- **Label**
  - Type: Binary
  - Description**:** A binary classification variable indicating whether the connection is normal (0) or attack (1).

Network Traffic Behaviour Variables

- **dur**
  - Type: Float
  - Description: Record total duration / Duration of the connection (in seconds).
- **sbytes**
  - Type: Integer
  - Description: Source to destination transaction bytes
- **dbytes**
  - Type: Integer
  - Description: Destination to source transaction bytes

Protocol and Connection State Variables

- **proto**
  - Type: Nominal
  - Description: Transaction / Network protocol used (e.g., tcp, udp, others).
- **state**
  - Type: Nominal
  - Description: Indicates the connection state and its dependent protocol flags (e.g., FIN, INT, REQ).

TTL and Flow State Variables

- **sttl**

    - Type: Integer
    - Description: Time-to-live value from source to destination.
- **dttl**
    - Type: Integer
    - Description: Time-to-live value from destination to source.

All in all, these variables capture the behaviour of packets size, flow directionality, protocol-level information, TTL characteristics, and connection lifecycle states. They allow the implementation of a variety of analysis techniques.

1.2 Assumptions

Outliers

- Outliers in variables such as dur, sbytes and dbytes are assumed to be valid reflections of abnormal or attack-related behaviour.
- Outliers that clearly violate logical rules will be removed.

Missing / Invalid Values

- Zero values in byte counts are assumed to be allowable for short or failed connections.

Categorical Variables (proto, state)

- Rare categories are assumed to be valid and will be grouped or merged only if they cause statistical imbalance.

TTL Variables (sttl, dttl)

- TTL values outside the normal IPv4 range (0–255), if any, will be treated as invalid and removed.
- Minor TTL anomalies are assumed to be typical of routing behaviour.

Binary Indicator (label)

- Assumed to be correctly labelled with no ambiguous values.
- Any unexpected values (not 0 or 1) will be filtered out.

Independence of Observations

- Each row is assumed to represent an independent network flow. Duplicate rows, if detected, will be removed.

1.3 Hypothesis

The network attacks in the dataset show statistically significant differences in the duration of connection (dur) and data transfer properties (sbytes, dbytes) when compared to normal traffic. Moreover, we anticipate that these attacks will be differentiated by TTL values (sttl, dttl), protocol type (proto), and connection state (state).

1.4 Objective

To investigate the impact of connection duration (dur) and transaction bytes (sbytes) on the presence of attack.

2.0 Data Preparation

2.1 Data import

```
# ==========================================
# DATA CLEANING AND PREPARATION
# ==========================================

# ---- STEP 1: Importing dataset ----

fileUrl = "C:\\[redacted]\\UNSW-NB15_uncleaned.csv"

dataset = read.csv(fileUrl, stringsAsFactors = F)
View(dataset)
```

Figure 2.1.1: Importing dataset csv file

The uncleaned UNSW-NB15 dataset was imported into R Studio from the local directory using read.csv(), with the prevention of string variables automatically converting into factors.

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	dload	sloss	dloss	sinpkt
1	1	0.121478	tcp	-	FIN	6	4	258	172	74.08749	252	254?	14158.94238	8495.365234	0_	0	24.2956
2	2	0.649902?	tcp	-	FIN	14	38	734	42014_	78.473372	62	252	8395.112305	503571.3125	2	17	49.915
3	3?	1.623129	tcp	-	FIN	8?	16?	364	13186	14.170161	62?	252	1572.271851	60929.23047	1	6	231.8755
4	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108?	62	252	2740.178955	3358.62207?	1	3?	152.8765
5	5	0.449454	tcp	-	FIN	10?	6	534	268	33.373826	254_	252	8561.499023	3987.059814_	2	1	47.75033
6	6	0.380537	tcp	-	FIN	10	6	534	268	39.41798_	254	252	10112.02539	4709.134766	2	1	39.92877
7	7?	0.637109	tcp	-	FIN	10	8	534	354	26.683033_	254_	252	6039.783203	3892.58374	2	1	68.26777
8	8	0.521584_	tcp	-	FIN	10	8	534	354	32.593026	254?	252	7377.527344	4754.74707	2	1_	55.794
9	9	0.542905	tcp	NA	FIN	10	8	534_	354	NA?	254_	252	7087.796387	4568.018555	2	1_	60.21088
10	10	0.258687_	tcp	-	FIN	10?	6	534	268?	57.985135	254	252_	14875.12012	6927.291016?	2	1_	27.50511
11	11	0.304853?	tcp	-	FIN	12	6	4142	268_	55.764583	254	252_	99641.46875	5878.243164	3	1	25.94881
12	12	2.093085_	tcp	smtp	FIN	62?	28	56329?	2212	42.520967_	62	252	211825.125	8152.559082?	28	8	34.31286
13	13_	0.416952	tcp	-	FIN	10	6	534	268	35.975363	254	252	9228.879883	4297.856445	2	1?	45.08877
14	14	0.996221	tcp	-	FIN	10	8_	564?	354	17.064487	254	252_	4079.416016?	2489.407471	2	1	110.6912
15	15	0.576755	tcp	-	FIN	10?	8	534	354	29.475254	254	252_	6671.810547_	4299.919434	2	1	64.08388
16	16	2e-06_	udp	snmp	INT	2	0?	138	0?	500000.0013	254	0?	2.76e+08	0	0?	0	0.002
17	17_	0.728252_	tcp	-	FIN	10_	6	534?	268	20.597266?	254	252	5283.885254	2460.686768	2	NA	80.91688
18	18	0.393556	tcp	http	FIN	10	8	860_	1096	43.195886_	62?	252	15733.46582	19494.04883	2	2	43.72844
19	19	0.387852_	tcp	-	FIN	10	6	534	268	38.674545?	254	252	9921.30957	4620.318848	2	1	38.11966
20	20	0.53784	tcp	-	FIN	10_	8	534	354	31.607913	254	252	7154.543945	4611.036621	2_	1	59.66944
21	21	0.23372	tcp	-	FIN	10	6	534	268	64.179359	254_	252	16464.14453	7667.293945	2	1	25.96888
22	22	0.338017	tcp	http	FIN	10	6	998	268?	44.376468	254	252	21277.03711	5301.508789	NA?	1	35.21811
23	23?	0.964656	tcp	ftp	CON	14	12	690	950_	25.915974	62	252	5315.884766	7223.300293	5	6	NA_
24	24	0.449703?	tcp	-	FIN	10	6	738	NA?	33.355347	254?	252_	11830.03027	3984.852051	2	1	42.61122
25	25_	0.921025	tcp	-	FIN	10	6	534_	268	16.286203	254	252_	4177.954102	1945.658447_	2	1	98.92255

Showing 1 to 25 of 175,341 entries, 45 total columns

Figure 2.1.2: Viewing the imported dataset

2.2 Cleaning / Pre-processing

```
# ==========================================
# CLEANING PROCESS FOR SELECTED VARIABLES
# ==========================================

# ---- STEP 2: Inspecting structure and summary ----

str(dataset)
summary(dataset[c("dur", "sbytes", "proto", "state", "dbytes", "sttl", "dttl", "label")

# ---- STEP 3: Removing question marks and underscores ----

dataset$dur <- trimws(gsub("[?_]", "", dataset$dur))
dataset$sbytes <- trimws(gsub("[?_]", "", dataset$sbytes))
dataset$label <- trimws(gsub("[?_]", "", dataset$label))
dataset$proto  <- trimws(gsub("[?_]", "", dataset$proto))
dataset$state  <- trimws(gsub("[?_]", "", dataset$state))
dataset$dbytes <- trimws(gsub("[?_]", "", dataset$dbytes))
dataset$sttl   <- trimws(gsub("[?_]", "", dataset$sttl))
dataset$dttl   <- trimws(gsub("[?_]", "", dataset$dttl))

# ---- STEP 4: Converting data types ----

# Converting continuous variables to numeric
dataset$dur <- as.numeric(dataset$dur)
dataset$sbytes <- as.numeric(dataset$sbytes)
dataset$dbytes <- as.numeric(dataset$dbytes)
dataset$sttl   <- as.numeric(dataset$sttl)
dataset$dttl   <- as.numeric(dataset$dttl)

# Converting categorical variables to factor
dataset$label <- as.factor(dataset$label)
dataset$proto <- as.factor(dataset$proto)
dataset$state <- as.factor(dataset$state)
```

Figure 2.2.1: Removing unwanted characters and converting data types

The structure and summary functions were used to inspect the data types, variable ranges and missing / irregular values for the selected variables dur, sbytes, proto, state, dbytes, sttl, dttl and label.

```
> summary(dataset[c("dur", "sbytes", "proto", "state", "dbytes", "sttl", "dttl", "label")])
     dur               sbytes             proto              state              dbytes             sttl
 Length:175341      Length:175341      Length:175341      Length:175341      Length:175341      Length:175341
 Class :character   Class :character   Class :character   Class :character   Class :character   Class :character
 Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character
     dttl               label
 Length:175341      Length:175341
 Class :character   Class :character
 Mode  :character   Mode  :character
```

Figure 2.2.2: Inspecting the summary of selected variables

After inspection, any unwanted inconsistent characters such as question marks and underscores within cells were removed to ensure consistent continuous and categorical values, while also retaining numbers that are valid scientific notations such as 1.1e-05. Afterwards, the continuous variables dur , sbytes, dbytes, sttl and dttl were converted to the numeric data type, while the categorical variables label, proto and state were converted to a factor, so that categorical classes (such as normal = 0 and attack = 1 for label) could be distinguished.

```r
# ---- STEP 5: Handling missing (NA) values ----

# Set NA values properly
dataset$dur[dataset$dur %in% c("NA", "")] <- NA
dataset$sbytes[dataset$sbytes %in% c("NA", "")] <- NA
dataset$label[dataset$label %in% c("NA", "")] <- NA
dataset$dbytes[dataset$dbytes %in% c("", "NA")] <- NA
dataset$sttl[dataset$sttl %in% c("", "NA")] <- NA
dataset$dttl[dataset$dttl %in% c("", "NA")] <- NA
dataset$proto[dataset$proto %in% c("", "NA")] <- NA
dataset$state[dataset$state %in% c("", "NA")] <- NA

# Displaying total NAs per column
colSums(is.na(dataset[c("dur", "sbytes", "proto", "state", "dbytes", "sttl", "dttl", "label")]))

# Using median imputations for numeric columns
dataset$dur[is.na(dataset$dur)] <- median(dataset$dur, na.rm = TRUE)
dataset$sbytes[is.na(dataset$sbytes)] <- median(dataset$sbytes, na.rm = TRUE)
dataset$dbytes[is.na(dataset$dbytes)] <- median(dataset$dbytes, na.rm = TRUE)
dataset$sttl[is.na(dataset$sttl)] <- median(dataset$sttl, na.rm = TRUE)
dataset$dttl[is.na(dataset$dttl)] <- median(dataset$dttl, na.rm = TRUE)

# Mode function
getMode <- function(v) {
  uniqv <- unique(v)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}

# Mode imputation for categorical
dataset$proto[is.na(dataset$proto)] <- getMode(dataset$proto)
dataset$state[is.na(dataset$state)] <- getMode(dataset$state)

# Removing rows with missing label
dataset <- dataset[!is.na(dataset$label) & dataset$label != "", ]
```

Figure 2.2.3: Handling NA values, and using Median and Mode imputation

Blank and missing values within the selected variables were converted to proper NA values before applying median imputation for the numerical variables and mode imputation for the categorical variables proto and state, and removing records with NA labels for data integrity.

It is also worth noting that no removal or capping was applied to some extreme records involving variables such as dur and sbytes in order to preserve realistic network behaviour and potential attack patterns. Such values could signify legitimate high-speed / high-volume attack flows, like botnets which are commonly used in large-scale attacks and transfer large volums of data to overwhelm or steal from targets (Scriptonet, 2025).

```r
# ---- STEP 6: Collapsing infrequent categories ----

# Displaying all categories
table(dataset$proto)
table(dataset$state)

# Importing required library
library(forcats)

# Collapsing proto: treating levels with < 500 occurrences as "OTHER"
dataset$proto <- fct_lump_min(dataset$proto, min = 500)

# Collapsing state: treating levels with < 1000 occurrences as "OTHER"
dataset$state <- fct_lump_min(dataset$state, min = 1000)
```

Figure 2.2.4: Script for displaying categories, importing library, and collapsing infrequent categories

Afterwards, with the intention of reducing cardinality and instability in advanced analysis techniques, Feature engineering (factor lumping by minimum) was applied to both categorical features proto and state, which groups low-frequency categories into “Other” using the forcats library. All levels in proto that had less than 500 occurrences were collapsed, whereas state levels with less than 1000 occurrences were collapsed. Thus, proto was reduced from 133 categories to 6, and state from 9 categories to 5.

```
> table(dataset$proto)

       3pc       a/n  aes-sp3-d       any     argus      aris       arp    ax.25   bbn-rcc       bna
        99        99       100       298        93        98      2794        98        98        98
br-sat-mon       cbt      cftp     chaos compaq-peer      cphb      cpnx      crtp     crudp       dcn
       100        95        99        98        99        99        97        96       100        95
       ddp       ddx       dgp       egp     eigrp     emcon     encap   etherip        fc      fire
        98        97        98        94        97        96        96        96        98        99
       ggp      gmtp       gre       hmp    i-nlsp      iatp        ib      icmp      idpr  idpr-cmtp
        98        97       221        96        98        96        99        14        97        96
      idrp      ifmp      igmp       igp        il        ip    ipcomp      ipcv      ipip      iplt
        99        96        17        97        98        97        98        99       100        98
     ipnip      ippc      ipv6  ipv6-frag   ipv6-no  ipv6-opts ipv6-route  ipx-n-ip      irtp      isis
        97        97       196        98        98        98        96        97        95        97
    iso-ip    iso-tp4  kryptolan      l2tp      larp    leaf-1    leaf-2 merit-inp   mfe-nsp      mhrp
        98        98        97        98        99        97        96       100        94        99
      micp    mobile       mtp       mux      narp    netblt nsfnet-igp       nvp      ospf       pgm
        99       195        97        96        99        96        99        95      2546        99
       pim      pipe      pnni   pri-enc       prm       ptp       pup       pvp       qnx       rdp
       195        98        97        98        96        99        97        99        97        96
      rsvp       rtp       rvd  sat-expak   sat-mon  sccopmce      scps      sctp      sdrp secure-vmtp
       195         1       100       100        98        99        95      1126        98        97
       sep      skip        sm       smp       snp sprite-rpc       sps       srp       st2       stp
       188        97       100        99        99        96        96       100        98        99
    sun-nd     swipe       tcf       tcp      tlsp      tp++   trunk-1   trunk-2       ttp       udp
       196       195        99     78287        98        97        98        97       100     62016
      unas       uti     vines      visa      vmtp      vrrp  wb-expak   wb-mon       wsn      xnet
     11860        99        99        97        97        97        99       100        96        98
   xns-idp       xtp      zero
        93       100        98
> table(dataset$state)

  CON   ECO   FIN   INT    no   PAR   REQ   RST   URN
12914    12 76327 80621     1     1  1952    83     1
```

Figure 2.2.5: Showcase of the proto & state categories before collapsing

```
> table(dataset$proto)

 ospf  sctp   tcp   udp  unas Other
 2488  1109 80142 60770 11617 13133
> table(dataset$state)

  CON   FIN   INT   REQ Other
12652 74783 79820  1910    94
```

Figure 2.2.6: Showcase of the proto & state categories after collapsing

2.3 Data Validation

```r
# ---- STEP 7: Validating data integrity ----

# Checking for invalid or impossible values
summary(dataset[c("dur", "sbytes", "dbytes", "sttl", "dttl")])

# Numerical variables must be non-negative
any(dataset$dur < 0)
any(dataset$sbytes < 0)
any(dataset$dbytes < 0)
any(dataset$sttl < 0)
any(dataset$dttl < 0)

# proto and state must have "Other" category
table(dataset$proto)
table(dataset$state)

# label must contain only 0 and 1
table(dataset$label)

# ---- STEP 8: Final verification and summary ----
summary(dataset[c("dur", "sbytes", "proto", "state", "dbytes", "sttl", "dttl", "label")])

# Saving cleaned dataset
write.csv(dataset, "UNSW-NB15_cleaned_subset_final.csv", row.names = FALSE)

# ==========================================
# END OF CLEANING PROCESS
# ==========================================
```

Figure 2.3.1: Script for data validation

In order to confirm that the cleaned dataset contained no negative or impossible values, the proto and state fields had the “Other” category, and that the label field only contained 0 and 1 categories, summary statistics along with logical checks were implemented. Ultimately, a final summary review of all the selected variables was performed to verify the validity of the cleaning approach, and the cleaned dataset was exported.

```
> any(dataset$dur < 0)
[1] FALSE
> any(dataset$sbytes < 0)
[1] FALSE
> any(dataset$dbytes < 0)
[1] FALSE
> any(dataset$sttl < 0)
[1] FALSE
> any(dataset$dttl < 0)
[1] FALSE
```

Figure 2.3.2: Checking for negative values

```
> table(dataset$label)

     0      1     NA
 52326 116933      0
```

Figure 2.3.3: Verifying label categories

```
> summary(dataset[c("dur", "sbytes", "proto", "state", "dbytes", "sttl", "dttl", "label")])
      dur                sbytes            proto          state           dbytes                sttl             dttl
 Min.   : 0.000001   Min.   :      46   ospf : 2488   CON  :12652   Min.   :       0   Min.   :  0.0   Min.   :  0.00
 1st Qu.: 0.000008   1st Qu.:     130   sctp : 1109   FIN  :74783   1st Qu.:       0   1st Qu.: 62.0   1st Qu.:  0.00
 Median : 0.001547   Median :     436   tcp  :80142   INT  :79820   Median :     164   Median :254.0   Median : 29.00
 Mean   : 1.350878   Mean   :    8888   udp  :60770   REQ  : 1910   Mean   :   14884   Mean   :183.7   Mean   : 79.74
 3rd Qu.: 0.662509   3rd Qu.:    1390   unas :11617   Other:   94   3rd Qu.:    1076   3rd Qu.:254.0   3rd Qu.:252.00
 Max.   :59.999989   Max.   :12965233   Other:13133                 Max.   :14655550   Max.   :255.0   Max.   :254.00
 label
 0 : 52326
 1 :116933
 NA:     0
```

Figure 2.3.4: Final verification of selected variables

3.0 Data Analysis

```
# ==========================================
# DATA ANALYSIS
# ==========================================

# Importing required libraries
library(dplyr)          # for data manipulation
library(ggplot2)        # for visualization
library(reshape2)       # for data reshaping
library(caret)          # for evaluating model performance
library(caTools)        # for data splitting
library(randomForest)   # for building random forest models
library(VIM)            # for handling missing data
```

Figure 3.1: Importing libraries required for data analysis

For data analysis, the libraries dplyr, ggplot2, reshape2, caret, and caTools, randomForest and VIM were imported.

3.1 ANALYSIS 3-1: What is the distribution of connection duration and source bytes across normal and attack traffic?

Descriptive Analysis

The following section shows the distribution of connection duration (dur) and source bytes (sbytes) across normal (label 0) and attack (label 1) traffic. Although summary statistics provide the general picture of variability and central tendencies, the detailed information about the underlying patterns and the differences between the two classes was offered with the help of the histogram, box plots, and density plot. Their outliers were retained in order to preserve realistic attack patterns.

Summary Statistics for dur and sbytes:

```
# ---------------------------------
# ANALYSIS 3-1: DESCRIPTIVE ANALYSIS
# ---------------------------------

# Summary Statistics
summary(dataset[, c("dur", "sbytes")])
dataset %>%
  group_by(label) %>%
  summarise(
    mean_dur = mean(dur, na.rm = TRUE),
    median_dur = median(dur, na.rm = TRUE),
    mean_sbytes = mean(sbytes, na.rm = TRUE),
    median_sbytes = median(sbytes, na.rm = TRUE),
    count = n()
  )
```

Figure 3.1.1: Script for summary statistics for dur & sbytes.

```
> summary(dataset[, c("dur", "sbytes")])
      dur               sbytes
 Min.   : 0.000000   Min.   :      28
 1st Qu.: 0.000008   1st Qu.:     114
 Median : 0.001547   Median :     436
 Mean   : 1.330759   Mean   :    8756
 3rd Qu.: 0.649596   3rd Qu.:    1328
 Max.   :59.999989   Max.   :12965233
> dataset %>%
+   group_by(label) %>%
+   summarise(
+     mean_dur = mean(dur, na.rm = TRUE),
+     median_dur = median(dur, na.rm = TRUE),
+     mean_sbytes = mean(sbytes, na.rm = TRUE),
+     median_sbytes = median(sbytes, na.rm = TRUE),
+     count = n()
+   )
# A tibble: 2 x 6
  label mean_dur median_dur mean_sbytes median_sbytes  count
  <fct>    <dbl>      <dbl>       <dbl>         <dbl>  <int>
1 0        0.990   0.0359         4043.          1282  54858
2 1        1.49    0.000009      10967.           200 116960
```

Figure 3.1.2: Output of summary statistics for sbytes & dur.

The summary statistics of both dur and sbytes were highly right-skewed. Right-skewed distributions are extremely common in real-world scenarios, particularly in website visit durations (Kutz, 2025). The mean value of 1.33 seconds is significantly greater than the median value of 0.0015 seconds, demonstrating that the long-duration outliers have a significant impact. Subsequently, extreme skewness is apparent in sbytes, with a mean of 8756 bytes as opposed to a median of 436 bytes. On grouping by label, it is observed that longer mean durations (1.49 secs vs. 0.990 secs) and considerably higher mean byte counts

(10967 bytes vs. 4043 bytes) are observed from attack traffic. However, for normal traffic, the median byte count is higher (1282 bytes vs 200 bytes), suggesting that attacks often involve occasional transfers of high volume alongside connections with low bytes.

Histogram for Connection Duration:

```
# Histogram dur
ggplot(dataset, aes(x = dur, fill = label)) +
  geom_histogram(bins = 60, alpha = 0.7, position = "identity") +
  labs(title = "Distribution of Connection Duration",
       x = "Duration (seconds)", y = "Count")
```

Figure 3.1.3: Script to show histogram for dur.

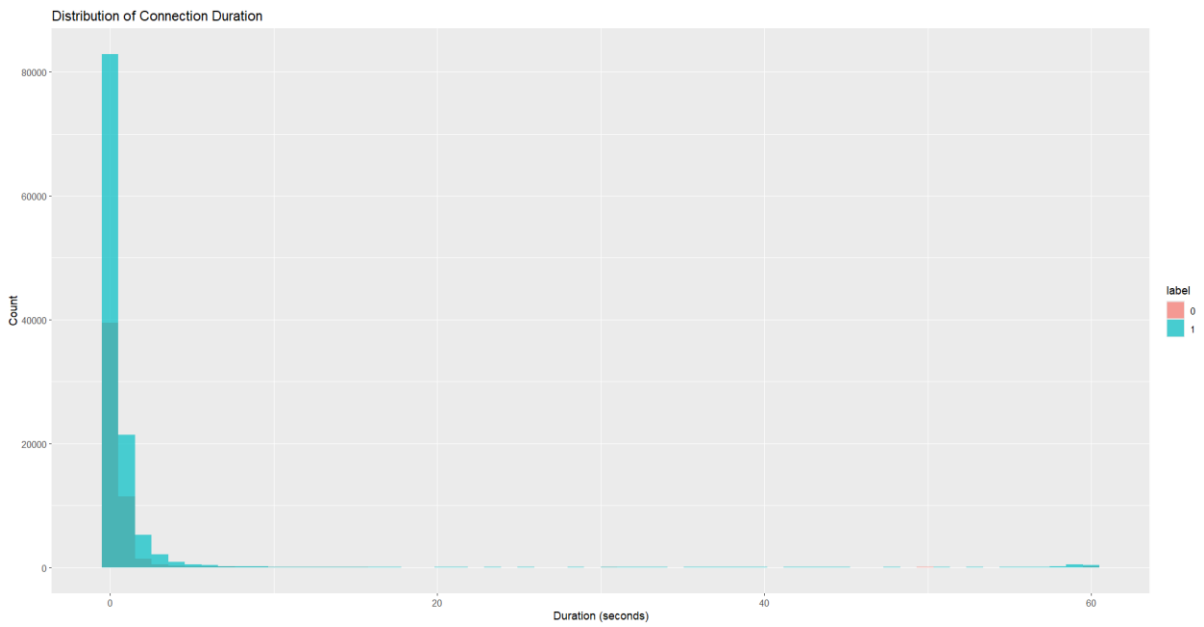

Figure 3.1.4: Histogram for Connection Duration.

The histogram for dur shows a greatly right-skewed distribution for both normal and attack traffic, as most connections near zero seconds. In a plot for right-skewed distributions, you will typically see data clustered to the left side, with the right tail extending further (GeeksForGeeks, 2025). This depicts a frequency of network interactions such as HTTP requests or DNS queries that are brief. While both classes reveal this pattern, a slightly heavier tail is seen from attack traffic which signifies an inclination towards longer durations. This could indicate slow reconnaissance or extended malicious sessions. Legitimate prolonged sessions or elaborate attacks are likely to be represented by outliers beyond 60 seconds.

Boxplots for Connection Duration and Source Bytes:

```
# Box Plot dur
ggplot(dataset, aes(x = label, y = dur, fill = label)) +
  geom_boxplot(outlier.alpha = 0.3) +
  labs(title = "Connection Duration by Attack Presence",
       x = "Label (0 = Normal, 1 = Attack)", y = "Duration (seconds)")
```

Figure 3.1.5: Script to show boxplot for dur.

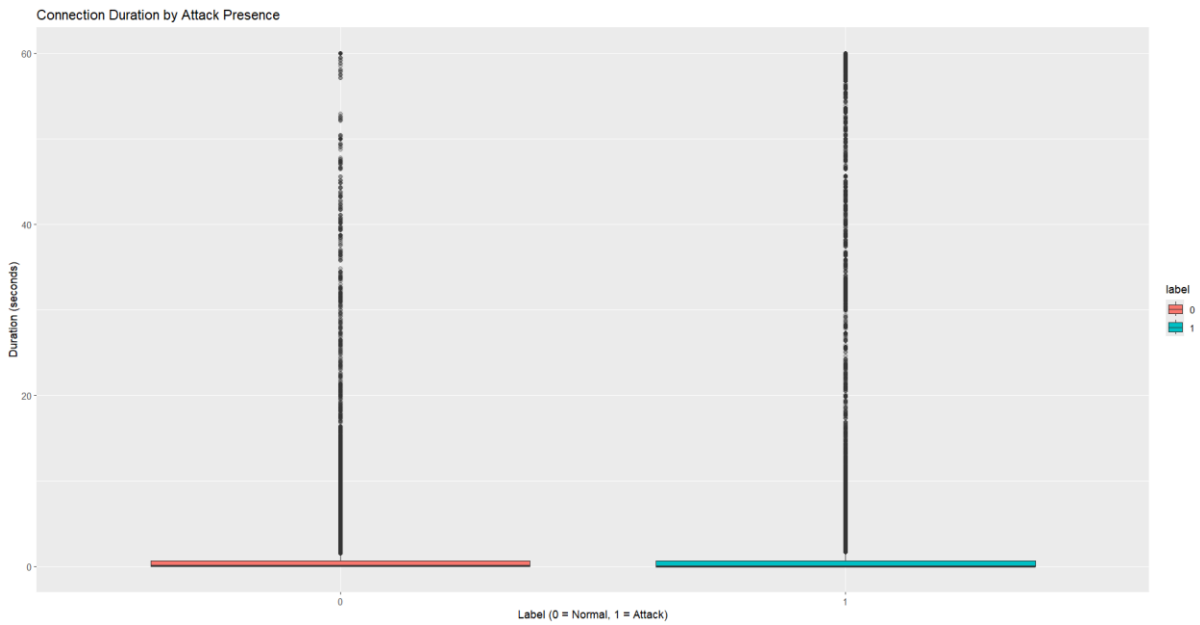

Figure 3.1.6: Boxplot for Connection Duration.

The extreme right-skewness of the histogram is verified in the dur box plot. The large number of connections with near zero second durations cause the interquartile range (IQR) and median to collapse into a thin horizontal line at the bottom of the plot, rendering the differences in the central disposition or spread almost identical from the boxes themselves. Nevertheless, the distribution of outliers emphasizes the key insight. Box plots make it straightforward to identify outliers (Six Sigma Development Solutions Inc., 2024). Both classes display high value outliers up to 60 seconds, but attack traffic labelled 1 has slightly more density and vertical dispersion than normal traffic labelled 0. This confirms the earlier finding that while most connections are short-lived, attack traffic is more likely to experience long-duration events.

```
# Box Plot sbytes
ggplot(dataset, aes(x = label, y = sbytes, fill = label)) +
  geom_boxplot(outlier.alpha = 0.3) +
  labs(title = "Source Bytes by Attack Presence",
       x = "Label (0 = Normal, 1 = Attack)", y = "Source Bytes")
```

Figure 3.1.7: Script to show boxplot for sbytes.

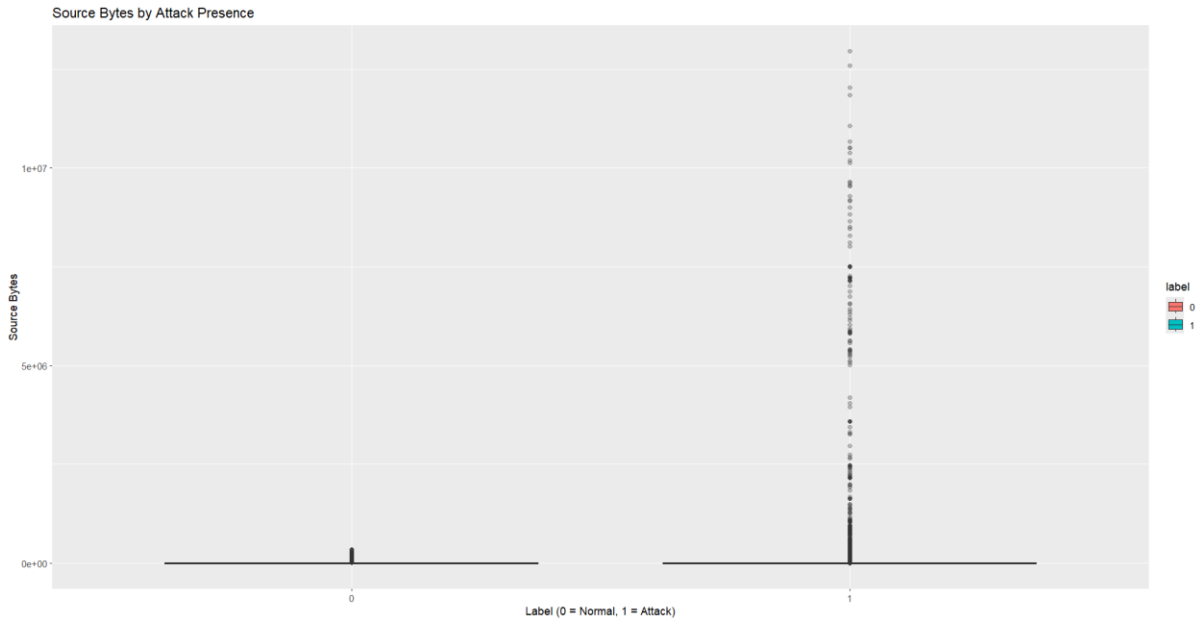

Figure 3.1.8: Boxplot for Source Bytes.

Similar to the previous plot, this box plot visualizes the extreme skewness of the sbytes variable as well. The IQR and median are compressed into a thin horizontal line at the bottom, representing that most of the connections involve very low data transfer volumes. The essential point lies in the outliers. Normal traffic only exhibits a limited cluster of low-value outliers. On the contrary, attack traffic has a dense, vertically stretched column of outliers, exceeding 10 million bytes. This provides insightful evidence that attacks consist of much larger data transfers, which makes sbytes an effective discriminative feature despite the lack of visible structure.

Density Plot for Source Bytes:

```
# Density Plot sbytes
ggplot(dataset, aes(x = sbytes, color = label, fill = label)) +
  geom_density(alpha = 0.3) +
  labs(title = "Density Distribution of Source Bytes by Attack Presence",
       x = "Source Bytes", y = "Density")
```

Figure 3.1.9: Script to show density plot for sbytes.

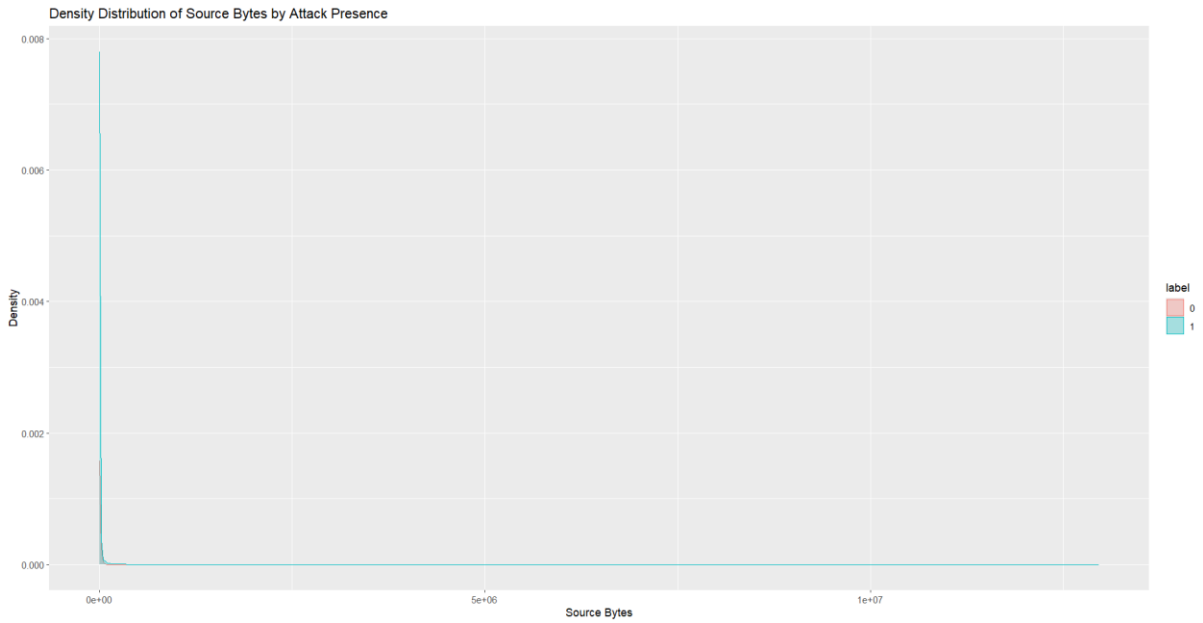

Figure 3.1.10: Density plot for Source Bytes.

The visuals from the density plot further solidify the contrasts observed in the box plot. Although they both have a peak value close to zero, attack traffic exhibits a secondary mode and a longer tail, which indicates an increased likelihood of large data transfers. The clear separation proves the use of sbytes to be a significant predictive feature.

To sum up, this descriptive analysis showcases different patterns in dur and sbytes distributions between normal and attack traffic. Both variables are occupied by short durations and low number of bytes, which is normal network behaviour. However, attack traffic is consistently proven to be longer in duration and with more bytes. These results prove that dur and sbytes are effective in capturing the distinctions in network behavior concerning malicious activity. These patterns were well understood through the visualizations and summary statistics, which guaranteed strong insights. The preservation of outliers was also an important factor in increasing the practical value of the analysis.

3.2 ANALYSIS 3-2: What is the relationship between connection duration and source bytes?

Correlation Analysis

This section discusses the correlation between connection duration (dur) and source bytes (sbytes) using a scatter plot, Spearman's rank correlation coefficient, and a correlation heatmap. These are techniques that offer a comprehensive understanding of how both variables interact, as well as their implications for differentiating between normal and attack traffic.

Scatter Plot:

```
# ---------------------------------
# ANALYSIS 3-2: CORRELATION ANALYSIS
# ---------------------------------

# Scatter Plot
ggplot(dataset, aes(x = dur, y = sbytes, color = label)) +
  geom_point(alpha = 0.4, size = 1) +
  labs(title = "Relationship between Duration and Source Bytes",
       x = "Duration", y = "Source Bytes")
```

Figure 3.2.1: Script to show scatter plot

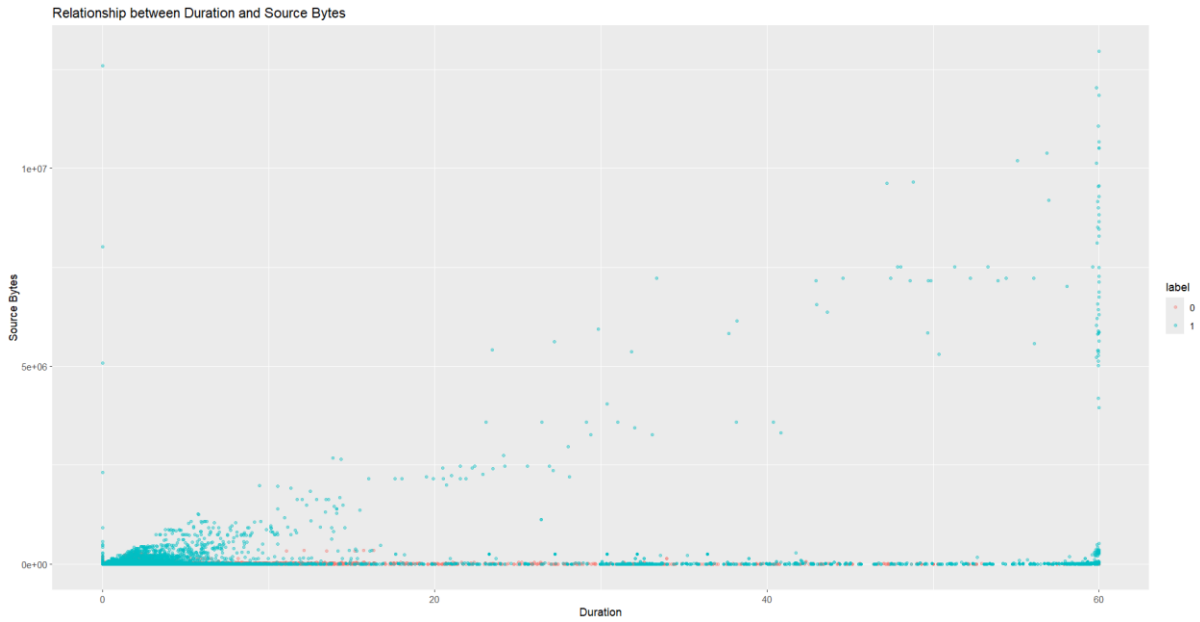

Figure 3.2.2: Scatter Plot for Duration and Source Bytes

The scatter plot indicates that there is a strong positive trend between dur and sbytes, and generally the larger the number of bytes, the longer the connections. Although the concentration of normal traffic is in the lower region, attack traffic extends into the upper right quadrant, suggesting a tendency for both prolonged durations and huge data transfers, consistent with patterns observed in DDoS attacks (Kaspersky, 2025). Thus, the notable outliers in this region often represent strong attacks.

Spearman's Correlation Coefficient:

```
# Spearman's Correlation Coefficient
correlation <- cor(dataset$dur, dataset$sbytes, method = "spearman", use = "complete.obs")
correlation
```

Figure 3.2.3: Script to show Spearman's Correlation Coefficient

```
> correlation <- cor(dataset$dur, dataset$sbytes, method = "spearman", use = "complete.obs")
> correlation
[1] 0.7473832
```

Figure 3.2.4: Output for Spearman's Correlation Coefficient

The Spearman's correlation coefficient calculation of 0.7473 confirms a strong positive monotonic relationship between both variables. This is a strong measure that takes into consideration the existence of outliers and non-normality to confirm the apparent trend. The result indicates that longer connections tend to involve more transfer of data, which is a pattern more prominent in attack traffic.

Correlation Heatmap:

```
# Correlation Heatmap
subset_data <- dataset[, c("dur", "sbytes")]
corr_matrix <- cor(subset_data, method = "spearman")
melted_corr <- melt(corr_matrix)

ggplot(melted_corr, aes(Var1, Var2, fill = value)) +
  geom_tile() +
  geom_text(aes(label = round(value, 2)), color = "white") +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
                       midpoint = 0, limit = c(-1, 1)) +
  labs(title = "Correlation Heatmap of Duration and Source Bytes",
       x = "", y = "")
```

Figure 3.2.5: Script to show Correlation Heatmap

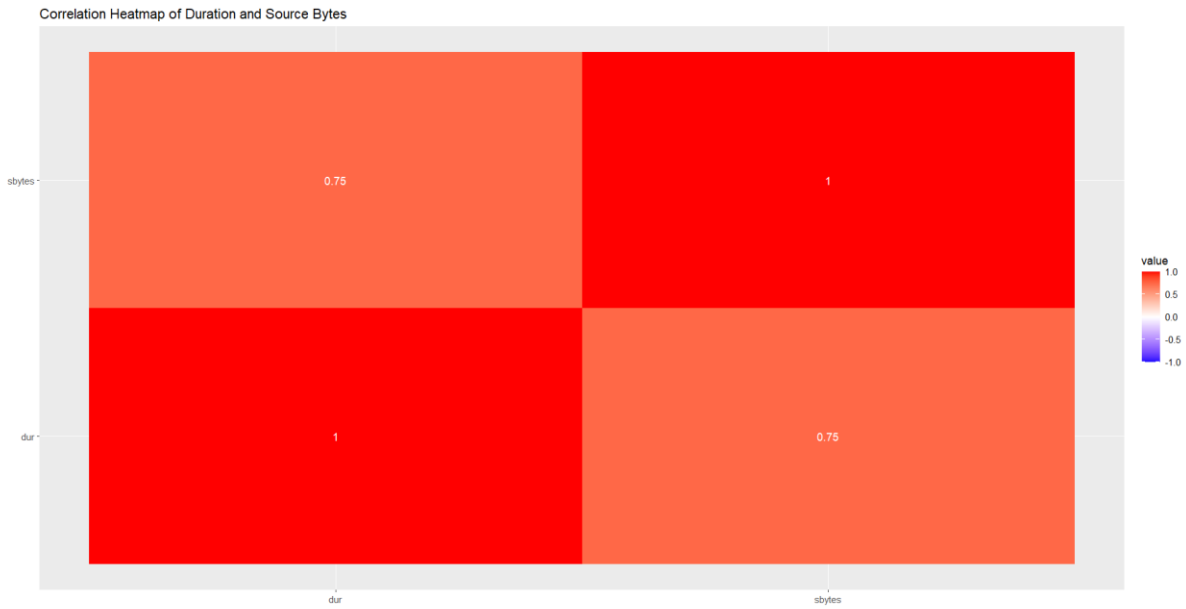

Figure 3.2.6: Correlation Heatmap for Duration and Source Bytes

Heatmap data visualization is a powerful tool used to represent numerical data graphically, where values are depicted using colors (GeeksForGeeks, 2024). The correlation heatmap serves as a visual summary of the correlation, and a red tile implies a high correlation of 0.75

between dur and sbytes. This sophisticated visualization method strengthens the earlier identified results of the scatter plot and correlation coefficient by offering a visualization of the relationship that is both descriptive and professional.

All in all, the relationship between dur and sbytes is strong, positive, and statistically significant. This is observed in the scatter plot's trend, the high Spearman's correlation coefficient, as well as the heatmap's comprehensive visualization. Attack traffic has a consistent combination of high byte counts and long durations, indicating the value of these variables for anomaly detection. In addition, the non-parametric methods and high-level visualization ensure clarity and reliability.

3.3 ANALYSIS 3-3: Is there a significant difference in connection duration and source bytes between attack and normal traffic?

Group Difference Analysis (Hypothesis Testing)

This section is related to whether connection duration (dur) and source bytes (sbytes) significantly differ between normal and attack traffic. To test this, Welch's two-sample t-tests and Wilcoxon rank-sum tests were applied. These methods examine the contrasts in means and medians, providing a clear statistical assessment of whether dur and sbytes show meaningful behavioural separation between attack and normal connections. T Test and Wilcoxon Test are two common tests used to compare two groups to determine if there is a significant difference between them (This vs. That, 2023).

T-tests (Mean Comparison):

```
# ------------------------------------------------------------
# ANALYSIS 3-3: GROUP DIFFERENCE ANALYSIS (HYPOTHESIS TESTING)
# ------------------------------------------------------------

# T-tests
t.test(dur ~ label, data = dataset)
t.test(sbytes ~ label, data = dataset)
```

Figure 3.3.1: Script to show T-Tests

```
> t.test(dur ~ label, data = dataset)

        Welch Two Sample t-test

data:  dur by label
t = -17.261, df = 150580, p-value < 2.2e-16
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 -0.5568839 -0.4433108
sample estimates:
mean in group 0 mean in group 1
      0.9903324       1.4904298

> t.test(sbytes ~ label, data = dataset)

        Welch Two Sample t-test

data:  sbytes by label
t = -11.164, df = 118375, p-value < 2.2e-16
alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
95 percent confidence interval:
 -8139.798 -5708.626
sample estimates:
mean in group 0 mean in group 1
       4042.944       10967.156
```

Figure 3.3.2: Output of T-Tests

Both variables had p-values below 2.2e-16 in the Welch two-sample t-tests that indicated significant differences in mean values of normal (dur: 0.990 s, sbytes: 4042 B) and attack connections (dur: 1.490 s, sbytes: 10967 B). These outputs reveal that attack flows on average transfer remarkably larger volumes of data and display differences in connection duration as opposed to normal traffic.

Wilcoxon Rank-Sum Tests (Median Comparison):

```
# Wilcoxon Rank-Sum Tests
wilcox.test(dur ~ label, data = dataset)
wilcox.test(sbytes ~ label, data = dataset)
```

Figure 3.3.3: Script to show Wilcoxon Rank-Sum Tests

```
> wilcox.test(dur ~ label, data = dataset)

        Wilcoxon rank sum test with continuity correction

data:  dur by label
W = 4135254819, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0

> wilcox.test(sbytes ~ label, data = dataset)

        Wilcoxon rank sum test with continuity correction

data:  sbytes by label
W = 4530202266, p-value < 2.2e-16
alternative hypothesis: true location shift is not equal to 0
```

Figure 3.3.4: Output for Wilcoxon Rank-Sum Tests

The non-parametric Wilcoxon tests have also provided p-values < 2.2e-16, supporting that the median values for both variables significantly differ between normal and attack traffic. This is of importance as both variables are right-skewed: The central tendency measure is stronger in medians. While attacks have much smaller median duration, normal flows have higher median sbytes, though the attack class has much higher mean because of a heavy tail of particularly large values. These findings altogether confirm the significant differences of the central distributions of dur and sbytes across classes.

In all group-difference methods in this section, the findings continuously show that connection duration (dur) and source bytes (sbytes) are significantly different between attack and normal traffic. The Welch t-tests yielded very small p-values (less than 2.2e-16), which proved the existence of strong mean differences, whereas Wilcoxon rank-sum tests detected equally significant median differences. These results indicate that there is generally more extreme or irregular behaviour in terms of duration and volume of bytes in attack flows. On the whole, Analysis 3-3 concludes that dur and sbytes show clear statistical separation between types of traffic, which offers a good foundation to use them as meaningful indicators.

3.4 ANALYSIS 3-4: Can connection duration, byte-flow characteristics, TTL behaviour, protocol type, and connection state collectively predict whether a network flow is an attack or normal traffic?

Predictive Analysis (Classification Modelling)

This section evaluates whether a combination of network-flow features - connection duration (dur), source and destination bytes (sbytes, dbytes), TTL values (sttl, dttl), protocol type (proto), and connection state (state) - can predict whether a network flow is an attack. Based on supervised machine-learning models (Logistic Regression and Random Forest), this analysis embodies multivariate complex patterns that differentiate between malicious and normal traffic. The section evaluates the combined discriminatory strength of these variables, as well as their appropriateness to automated attack detection, by training and testing these models on the prepared dataset.

Logistic Regression:

```
# ------------------------------------------------------------
# ANALYSIS 3-4: PREDICTIVE ANALYSIS (CLASSIFICATION MODELLING)
# ------------------------------------------------------------

# ---- LOGISTIC REGRESSION ----

# Train-Test Split
set.seed(123)
split <- sample.split(dataset$label, SplitRatio = 0.8)

training_set <- subset(dataset, split == TRUE)
test_set     <- subset(dataset, split == FALSE)

dim(training_set)
dim(test_set)

# Logistic Regression Model
classifier <- glm(label ~ dur + sbytes + dbytes + sttl + dttl + proto + state,
                  data = training_set,
                  family = binomial)

summary(classifier)

# Prediction
pred_prob_test <- predict(classifier, type = "response",
                          newdata = test_set)

pred_class_test <- ifelse(pred_prob_test > 0.5, "1", "0")
pred_class_test <- factor(pred_class_test, levels = c("0", "1"))
test_set$label  <- factor(test_set$label, levels = c("0","1"))

# Confusion Matrix
cm <- confusionMatrix(pred_class_test, test_set$label)
cm
```

Figure 3.4.1: Script for Logistic Regression

Logistic regression is a supervised machine-learning classification model that is applied to determine the likelihood of a network flow being an attack or normal class, based on various predictive variables. Dur, sbytes, dbytes, sttl, dttl, proto and state are used to construct the model and comprehend how the combination of these network-flow features leads to attack probability. The coefficient signs, z-values, p-values and confidence intervals are used to gain insight into the impact of each predictor, while the measures of accuracy and performance of the model assess the ability of the variables to classify network traffic.

Initial Attempt:

```
# Logistic Regression Model
classifier <- glm(label ~ dur + sbytes,
                  data = training_set,
                  family = binomial)

summary(classifier)
```

Figure 3.4.2: Initial Logistic Regression Model

Initially, for experimental purposes, logistic regression was only applied to the primary variables dur and sbytes.

```
> summary(classifier)

Call:
glm(formula = label ~ dur + sbytes, family = binomial, data = training_set)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) 7.863e-01  6.031e-03  130.38  < 2e-16 ***
dur         1.104e-02  1.065e-03   10.37  < 2e-16 ***
sbytes      7.123e-07  1.579e-07    4.51  6.5e-06 ***
---
Signif. codes:
0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 167476  on 135406  degrees of freedom
Residual deviance: 167272  on 135404  degrees of freedom
AIC: 167278

Number of Fisher Scoring iterations: 6
```

Figure 3.4.3: Coefficients for Initial Logistic Regression Model

Although the logistic regression model with only dur and sbytes had statistically significant coefficients ($p < 0.001$) indicating that both variables have measurable influence on the likelihood of a connection being classified as an attack, the practical effect sizes were very small, suggesting limited predictive power.

```
> cm <- confusionMatrix(pred_class_test, test_set$label)
> cm
Confusion Matrix and Statistics

          Reference
Prediction     0     1
         0     0     0
         1 10465 23387

               Accuracy : 0.6909
                 95% CI : (0.6859, 0.6958)
    No Information Rate : 0.6909
    P-Value [Acc > NIR] : 0.5026

                  Kappa : 0

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.0000
            Specificity : 1.0000
         Pos Pred Value :    NaN
         Neg Pred Value : 0.6909
             Prevalence : 0.3091
         Detection Rate : 0.0000
   Detection Prevalence : 0.0000
      Balanced Accuracy : 0.5000

       'Positive' Class : 0
```

Figure 3.4.4: Confusion Matrix and Statistics for Initial Logistic Regression Model

Confusion Matrix:

Prediction	Reference		
		0	**1**
	0	0	0
	1	10465	23387

The confusion matrix revealed that the model had a 69.1% accuracy with all test samples being classified as attacks. This matches the dataset's class distribution (majority attacks), therefore the model merely predicted the most common class. The sensitivity to detect normal traffic was 0% while specificity was 100% with Kappa = 0, confirming the lack of real discriminative ability. This indicates that dur and sbytes alone are insufficient for effective attack detection and additional predictors are required to improve performance.

Final Attempt:

```
# Logistic Regression Model
classifier <- glm(label ~ dur + sbytes + dbytes + sttl + dttl + proto + state,
                  data = training_set,
                  family = binomial)
```

Figure 3.4.5: Final Logistic Regression Model

Following the failure of the initial attempt, five additional predictors - dbytes, sttl, dttl, proto, and state - were added into the logistic regression model to determine whether these variables significantly influence the likelihood of a network flow being classified as an attack.

```
> summary(classifier)

Call:
glm(formula = label ~ dur + sbytes + dbytes + sttl + dttl + proto +
    state, family = binomial, data = training_set)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.560e+01  1.321e+02  -0.194   0.8463
dur          3.328e-02  2.637e-03  12.622  < 2e-16 ***
sbytes       4.863e-06  5.089e-07   9.556  < 2e-16 ***
dbytes       3.533e-07  8.792e-08   4.019 5.86e-05 ***
sttl         1.622e-03  1.334e-04  12.156  < 2e-16 ***
dttl         2.034e-02  1.814e-04 112.079  < 2e-16 ***
protoospf    2.266e+01  1.321e+02   0.172   0.8638
protosctp    4.090e+01  2.065e+02   0.198   0.8430
prototcp     1.847e+01  1.321e+02   0.140   0.8887
protoudp     2.108e+01  1.321e+02   0.160   0.8732
protounas    3.687e+01  1.484e+02   0.248   0.8038
protoOther   2.455e+01  1.321e+02   0.186   0.8525
stateFIN     3.101e+00  6.956e-02  44.585  < 2e-16 ***
stateINT     6.875e+00  7.182e-02  95.730  < 2e-16 ***
stateREQ     4.167e+00  1.293e-01  32.227  < 2e-16 ***
stateOther   1.281e+00  5.679e-01   2.255   0.0241 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 172181  on 137453  degrees of freedom
Residual deviance:  66094  on 137438  degrees of freedom
AIC: 66126

Number of Fisher Scoring iterations: 17
```

Figure 3.4.6: Coefficients for Final Logistic Regression Model

The new logistic regression model indicates that most of the predictors (duration, byte volumes, TTL values, and connection states) are statistically significant in determining attack probability ($p < 0.001$). Large reductions in deviance and a lowered AIC signify that the model excellently fits the data and captures significant differences between malicious and benign flows. TTL anomalies, byte asymmetries and abnormal connection states are behavioural indicators that greatly enhance the explainability of a model compared to dur and sbytes only.

```
> cm <- confusionMatrix(pred_class_test, test_set$label)
> cm
Confusion Matrix and Statistics

          Reference
Prediction     0     1
         0  8559   211
         1  2413 23181

               Accuracy : 0.9236
                 95% CI : (0.9208, 0.9264)
    No Information Rate : 0.6807
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.8144

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.7801
            Specificity : 0.9910
         Pos Pred Value : 0.9759
         Neg Pred Value : 0.9057
             Prevalence : 0.3193
         Detection Rate : 0.2491
   Detection Prevalence : 0.2552
      Balanced Accuracy : 0.8855

       'Positive' Class : 0
```

Figure 3.4.7: Confusion Matrix and Statistics for Final Logistic Regression Model

Confusion Matrix:

Prediction	**Reference**		
		0	**1**
	0	8559	211
	1	2413	23181

The improved model produced a high number of true positives and true negatives with relatively few false classifications, thus achieving 92.36% accuracy, which is very strong for a binary classifier in a cybersecurity dataset. It has great sensitivity (correctly finds 78.01% of normal traffic) and excellent specificity (correctly finds 99.10% of attack traffic), which demonstrates its reliability in detecting both flows. The Kappa value (0.8144) and balanced accuracy (0.8855) prove that the model maintains high discriminative power even under moderate class imbalance. These results conclude that the selected variables provide meaningful predictive power for classifying traffic.

Random Forest:

```
# ---- RANDOM FOREST ----

# Ensuring No NA Values
training_set <- hotdeck(training_set, domain_var = "label", imp_var = FALSE)
test_set     <- hotdeck(test_set, domain_var = "label", imp_var = FALSE)

training_set$label <- droplevels(training_set$label)
test_set$label <- droplevels(test_set$label)

levels(training_set$label)

# Random Forest Model
rf_model <- randomForest(
  label ~ dur + sbytes + dbytes + sttl + dttl + proto + state,
  data = training_set,
  ntree = 500
)

# Model Summary and Variable Importance Plot
rf_model
importance(rf_model)
varImpPlot(rf_model)

# Prediction
y_pred_rf <- predict(rf_model, newdata = test_set)

# Confusion Matrix
cm_rf <- confusionMatrix(table(y_pred_rf, test_set$label))
cm_rf
```

Figure 3.4.8: Script for Random Forest

Random Forest is a machine learning classifier which is ensemble-based and constructs multiple decision trees to combine their outputs and produce strong and accurate predictions. In this analysis, the Random Forest model is trained on the same set of features to learn complex nonlinear relationships and interaction patterns that could exist in the network data. This technique offers both the high predictive performance and detailed measures of variable

importance, which enable us to compare the strength of features in separating attack traffic and normal flows.

```
> rf_model

Call:
 randomForest(formula = label ~ dur + sbytes + dbytes + sttl +      dttl + proto + state, data = training_set, ntree = 500)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 6.24%
Confusion matrix:
      0     1 class.error
0 36136  7750  0.17659390
1   824 92744  0.00880643
```

Figure 3.4.9: Random Forest Model

The out-of-bag (OOB) error rate of this Random Forest Classifier was 6.24%, showing the high stability of the model during training.

```
> importance(rf_model)
        MeanDecreaseGini
dur             4840.195
sbytes          4572.312
dbytes          4694.325
sttl           20011.072
dttl            7991.725
proto           1742.393
state           4536.137
```

Figure 3.4.10: Importance of the selected variables

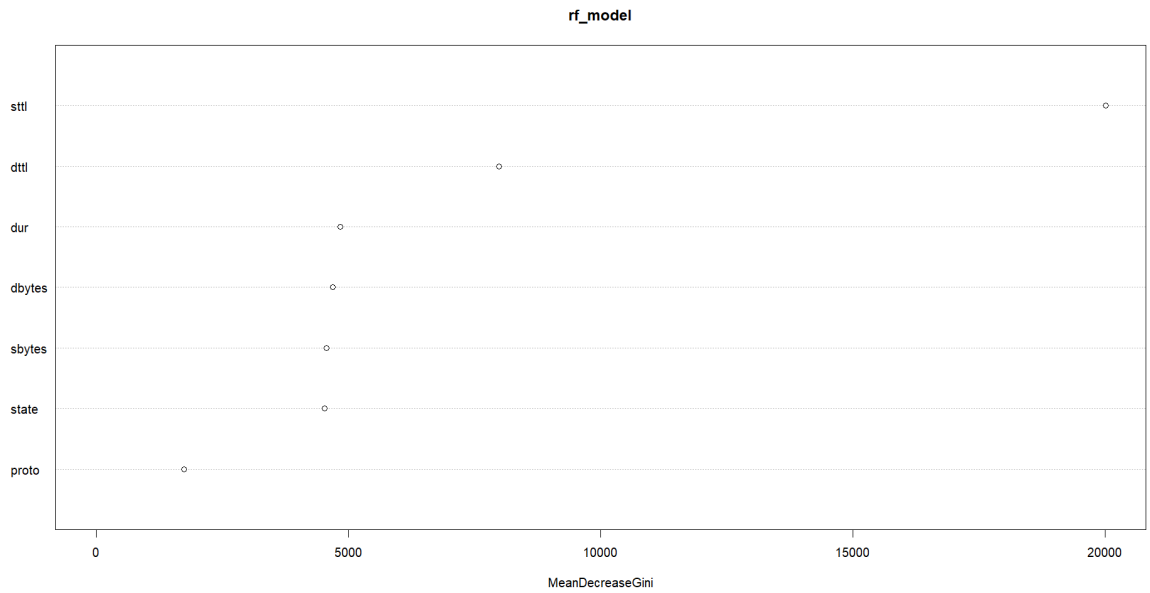

Figure 3.4.11: Variable Important Plot of selected variables

The variable importance results show that sttl, dttl, dur, dbytes, sbytes, and state all greatly contribute to the model's decision-making, with sttl being the best predictor. This implies that TTL behaviour, connection duration and byte-flow patterns encode significant differences between legitimate and malicious network connections.

```
> cm_rf
Confusion Matrix and Statistics


y_pred_rf     0     1
        0  9067   194
        1  1905 23198

               Accuracy : 0.9389
                 95% CI : (0.9363, 0.9414)
    No Information Rate : 0.6807
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.8534

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.8264
            Specificity : 0.9917
         Pos Pred Value : 0.9791
         Neg Pred Value : 0.9241
             Prevalence : 0.3193
         Detection Rate : 0.2639
   Detection Prevalence : 0.2695
      Balanced Accuracy : 0.9090

       'Positive' Class : 0
```

Figure 3.4.12: Confusion Matrix and Statistics for Random Forest Model

Confusion Matrix:

Prediction	**Reference**		
		0	**1**
	0	9067	194
	1	1905	23198

The Random Forest confusion matrix displays even better results, with extremely high true positive and true negative values, and lower false positive and false negative values than logistic regression. The model performed well on the test set and achieved 93.89% accuracy, with excellent specificity (99.17%) and strong sensitivity (82.64%), which means that it can detect most attack flows and has very low false-alarm rates. The high Kappa (0.8534) and balanced accuracy (0.9090) are indicative of excellent agreement beyond chance and high performance. These findings confirm the great predictive capability of the selected variables for attack classification.

Both machine-learning models that the chosen variables, particularly, TTL values, byte flow asymmetries, connection states, and the core variables dur and sbytes form a good predictive foundation for detecting malicious traffic. Logistic Regression achieved 92.36% accuracy while the Random Forest achieved an even higher accuracy of 93.89%, emphasizing sttl, dttl, dur and variables related to bytes as significant contributors, and demonstrating superior sensitivity and near-perfect specificity to validate the impact of these variables on the likelihood of attacks. The combination of these findings demonstrates that multivariate patterns in these features are reliable in distinguishing between attack and normal flows, and that the variables selected are statistically significant and predictively powerful for modelling network attacks.

4.0 Conclusion

4.1 Overall discussion on the findings

The analyses conducted in this study consistently demonstrate that key network-flow characteristics can effectively differentiate between normal and malicious traffic. Statistical tests showed that connection duration (dur) and source bytes (sbytes) differ significantly between attack and normal flows, indicating distinct behavioural patterns in these variables. Building on these findings, the predictive models further confirmed that combining these variables with additional features such as destination bytes (dbytes), TTL values (sttl, dttl), protocol type (proto), and connection state (state) provides strong classification capability. Both logistic regression and Random Forest achieved high accuracy, with Random Forest showing slightly superior performance. Overall, the results indicate that meaningful patterns exist within network traffic data that can be leveraged to detect attacks reliably.

4.2 Recommendation

Based on the findings, it is recommended that network intrusion detection systems incorporate multiple network-flow attributes rather than relying on a single indicator. Features related to connection duration, byte-flow behaviour, TTL values, and connection states were shown to provide strong predictive value when combined in machine-learning models. In particular, ensemble approaches such as Random Forest may offer improved robustness and detection accuracy in real-world environments. Integrating these features into automated monitoring systems could enhance the ability of organisations to identify abnormal network activity and respond to potential threats more efficiently.

4.3 Limitation and future direction

Despite the promising results, several limitations should be acknowledged. The analysis was conducted using a single dataset, which may not fully represent the diversity of real-world network environments and attack patterns. Additionally, only two machine-learning models were explored, and the selected features represent only a subset of the available attributes in the dataset. Future research could evaluate additional models such as gradient boosting or deep learning techniques, as well as investigate feature selection methods to further improve detection performance. Testing the models on multiple datasets or real-time network data would also help validate their generalisability and practical applicability in operational cybersecurity systems.

5.0 References

GeeksForGeeks. (2024, June 12). *What is Heatmap Data Visualization and How to Use It?* https://www.geeksforgeeks.org/data-visualization/what-is-heatmap-data-visualization-and-how-to-use-it/

GeeksForGeeks. (2025, June 10). *Left skew vs Right skew.* https://www.geeksforgeeks.org/data-science/left-skew-vs-right-skew/

Kaspersky. (2025). *What is a DDoS Attack?* https://me-en.kaspersky.com/resource-center/threats/ddos-attacks

Kutz, J. (2025, September 30). *What is Skewed Data: Examples & Types.* AirByte. https://airbyte.com/data-engineering-resources/skewed-data

Scriptonet. (2025, May 26). *Botnet in Cyber Security: The Hidden Army Behind Modern Attacks.* https://www.scriptonet.com/journal/botnet-in-cyber-security/#:~:text=Botnets%20are%20also%20used%20to%20send%20massive%20volumes,information%20such%20as%20login%20credentials%20%26%20financial%20data.

Six Sigma Development Solutions Inc. (2024). *Using Box Plots to Understand Variability and Outliers in Data.* https://sixsigmadsi.com/box-plots/

This vs. That. (2023). *T Test vs. Wilcoxon Test.* https://thisvsthat.io/t-test-vs-wilcoxon-test